Hash Types

Every guide in this series so far has pointed at the same unresolved variable and deferred it here: your JtR and Hashcat guides both opened with "fast hashes vs slow hashes" as the single biggest determinant of cracking feasibility, and your Wordlists guide's entire thesis only pays off once you're actually running against a specific algorithm. This guide is that deferred piece — **what these algorithms actually are, structurally, and why some resist cracking by orders of magnitude more than others.** Worth locking in the distinction immediately: **Part 1 covers general-purpose hash functions** (MD5, SHA family, NTLM) — built for speed and integrity-checking, never intended to protect a secret against a determined attacker — **Part 2 covers password-hardening functions** (bcrypt, scrypt, Argon2, PBKDF2) — built specifically to be slow, memory-hungry, and cracking-resistant by design. Same input (a password), same output shape (a fixed-length digest) — completely different threat model each was designed against.

---

1. What a Hash Function Actually Guarantees — The Foundation

A cryptographic hash function takes an input of any size and produces a **fixed-length digest**, with three properties every hash type below shares:

```
1. Deterministic     → same input always produces the same output
2. One-way            → computationally infeasible to reverse the digest back to the i
nput directly
3. Collision-resistant  → infeasible to find two DIFFERENT inputs that produce the SAM
E digest
```

The single fact that separates Part 1 from Part 2 — none of those three properties say anything about SPEED, and speed is a completely independent design choice layered on top. A hash function can satisfy all three properties perfectly while still being trivially fast to compute billions of times per second — which is exactly what makes it a poor choice for password storage even though it's a perfectly good, secure hash function for its *originally intended* purpose (checksums, digital signatures, data integrity verification).

```
Original design intent of MD5/SHA family → verify data hasn't been TAMPERED with (file
integrity, digital signatures) - here, SPEED is a desirable property, since you want t
o hash gigabytes of data quickly

Password storage's actual requirement → make each GUESS as expensive as possible for a
n attacker running millions of guesses - here, the EXACT SAME speed that made MD5 usef
ul for integrity checking makes it actively dangerous for password storage
```

This mismatch — using a fast, integrity-oriented hash for a secrecy-oriented purpose it was never designed for — is the single most common root cause underlying every "why was this crackable so fast" finding in a real engagement, and it's exactly what your Hashcat guide's benchmark numbers (Section 7) made concrete.

PART 1: General-Purpose (Fast) Hash Functions

2. MD5 — Still Everywhere, Despite Being Broken

Designed in 1991, produces a 128-bit (32 hex character) digest. Cryptographically **broken** for collision-resistance since 2004 (researchers can deliberately construct two different inputs with the same MD5 hash) — but remains extremely common in the wild for password storage in legacy systems, precisely because it's fast and was the default choice for decades before that changed.

```
Format:      32 hex characters, e.g. 5f4dcc3b5aa765d61d8327deb882cf99
Hashcat mode: -m 0
JtR format:   --format=raw-md5

Cracking speed: TENS OF BILLIONS of hashes/second on a modern GPU
                 (per your Hashcat guide's Section 7 benchmark table)
```

Collision-resistance being broken is actually a *separate* weakness from the cracking-speed problem covered here — for password storage specifically, MD5's danger is almost entirely about raw speed, not the collision attacks (which matter more for signature-forgery scenarios).

3. SHA-1 and the SHA-2 Family (SHA-256, SHA-512)

SHA-1 (160-bit digest) is also now considered broken for collision resistance (a practical collision was publicly demonstrated in 2017) and, like MD5, remains

fast and unsuited to password storage. SHA-2 (SHA-256, SHA-512) remains collision-resistant and cryptographically sound **for its intended integrity/signature use cases** — but shares the exact same speed problem for password storage specifically:

```
SHA-1:    40 hex characters   -m 100    --format=raw-sha1
SHA-256:  64 hex characters   -m 1400   --format=raw-sha256
SHA-512: 128 hex characters   -m 1700   --format=raw-sha512

Cracking speed: still BILLIONS of hashes/second on a modern GPU - the LONGER digest do
es NOT meaningfully slow down cracking, since digest length and computational cost per
guess are two entirely different properties
```

A genuinely common misconception worth flagging directly: "SHA-256 is more secure than MD5" is true for collision-resistance and cryptographic soundness, but **irrelevant** to password-cracking resistance — an unsalted SHA-256 password hash is cracked at nearly the same explosive speed as MD5, because neither was ever designed to be slow.

4. NTLM — Windows' Legacy Password Hash

Windows' hashing scheme for local/domain account passwords, still in wide use for backward compatibility despite Microsoft's own guidance recommending Kerberos over NTLM authentication where possible. NTLM is effectively **unsalted MD4** — an even older, faster, weaker algorithm than MD5:

```
Format:       32 hex characters (same visual shape as MD5, DIFFERENT
                algorithm underneath)
Hashcat mode: -m 1000
JtR format:   --format=nt

Obtained via: SAM database dumps, NTDS.dit extraction from a domain controller, or cap
tured NetNTLMv2 challenge-response hashes (Hashcat -m 5600) during network authenticat
ion
```

The **complete absence of salting** compounds the speed problem directly — every unsalted NTLM hash in a dumped set can be checked against a single precomputed candidate hash simultaneously, rather than needing to recompute per-user, making a large domain-controller dump exceptionally fast to process as a whole (a point that connects directly to Section 8's salting explanation below).

5. Kerberos Hashes — Kerberoasting Targets

Referenced in both your JtR guide (`krb5tgs`) and Hashcat guide (`-m 13100`) — Kerberos service ticket encryption uses a key derived from a service account's password, meaning a captured TGS ticket can be cracked offline exactly like any other password hash once extracted:

```
Hashcat mode: -m 13100  (Kerberos 5 TGS-REP etype 23)
JtR format:   --format=krb5tgs

Speed characteristics vary by Kerberos encryption type used - RC4 based tickets (etype
23) crack at speeds comparable to NTLM; AES based tickets (etype 17/18) are meaningful
ly slower and more resistant, since AES-based Kerberos key derivation adds real comput
ational cost per guess
```

6. When a Fast Hash Is Actually Fine — Not Every Use Case Is Password Storage

Worth stating explicitly, since this whole Part exists to explain a *specific* misuse: MD5/SHA-family hashes remain entirely appropriate for their originally intended purposes — file integrity checksums, deduplication, digital signature construction, HMAC constructions for message authentication. **The problem is exclusively when a fast general-purpose hash is used to store a SECRET (a password) that needs to resist a brute-force guessing attack** — that's a fundamentally different threat model than "detect accidental corruption" or "verify a signature," and Part 2 exists because someone eventually built algorithms specifically for that different threat model.

PART 2: Password-Hardening (Slow) Hash Functions

7. bcrypt — The Long-Standing Default

Designed specifically for password storage in 1999, built around the Blowfish cipher with a deliberately tunable **cost factor** that controls how many rounds of internal computation each hash requires:

```
Format:       $2a$ / $2b$ / $2y$ [cost]$[22-char salt][31-char hash]
              e.g. $2b$12$R9h/cIPz0gi.URNNX3kh2OPST9/PgBkqquzi.Ss7KIUgO2t0jWMUW

Hashcat mode: -m 3200
JtR format:   --format=bcrypt
```

```
Cracking speed: THOUSANDS to low TENS OF THOUSANDS of hashes/second on a modern GPU -
a roughly SIX ORDER OF MAGNITUDE gap versus MD5/SHA on identical hardware
```

```
The cost factor (the number after $2b$) is EXPONENTIAL:
  cost=10 → 2^10 = 1,024 internal rounds
  cost=12 → 2^12 = 4,096 internal rounds  (common current default)
  cost=14 → 2^14 = 16,384 internal rounds (higher security, slower for BOTH attacker a
nd legitimate server)
```

This tunability is bcrypt's core design idea — as hardware gets faster over time, the cost factor can simply be increased for newly-stored passwords, keeping the "time to crack" roughly constant even as attacker hardware improves.

8. Salting — Why It's Inseparable From This Discussion

Every properly-implemented password-hardening function embeds a **unique, random salt** per password, generated at hash time and stored alongside the hash itself:

```
Without salt:  identical passwords → IDENTICAL hashes
               → precomputed rainbow tables work instantly
               → cracking one hash effectively cracks every
                  matching duplicate in the entire dump at once

With salt:     identical passwords → COMPLETELY DIFFERENT hashes
               (salt is mixed in before hashing)
               → rainbow tables are useless
               → EVERY hash must be attacked individually, even
                  if two users happen to share the exact same password
```

bcrypt, scrypt, and Argon2 all build salting directly into their format by design (you can see the salt embedded in the hash string itself in Section 7's example) — this is a structural guarantee, not something a developer can accidentally forget when using these functions correctly, which is a real advantage over manually salting a fast hash like MD5 (which developers frequently do wrong or skip entirely).

9. scrypt — Adding Memory-Hardness

Designed in 2009 specifically to resist an attack vector bcrypt doesn't fully address: **GPU/ASIC parallelization**. bcrypt is compute-hard but relatively memory-light, meaning an attacker can run massively many parallel instances on cheap, memory-light hardware. scrypt deliberately requires large amounts

of memory **per guess**, which is expensive to parallelize at scale even with specialized hardware:

```
Hashcat mode: -m 8900 (or 9300 for the Drupal7 variant, 15700 for specific implementat
ions - scrypt has several format-specific hashcat modes depending on how it was implem
ented/wrapped)

Tunable parameters: N (CPU/memory cost), r (block size), p (parallelization factor) -
all three independently increase resistance at the cost of legitimate server-side hash
ing time
```

The memory-hardness concept here directly foreshadows Section 10's Argon2, which took this idea further and became the current recommended standard.

10. Argon2 — The Current Recommended Standard

Winner of the 2015 Password Hashing Competition, and the algorithm most current security guidance (OWASP included) recommends as the default choice for new systems. Comes in three variants, each tuned against a slightly different threat model:

```
Argon2d  → maximizes resistance to GPU cracking attacks (data-dependent memory access
- fastest, but theoretically more vulnerable to side-channel attacks)
Argon2i   → data-INdependent memory access, resistant to side-channel attacks, slightl
y weaker against pure GPU-cracking resistance than Argon2d
Argon2id   → HYBRID of both - the currently recommended default for password storage,
balancing both threat models

Hashcat mode: -m 34000 -m 22400 (varies by variant/encoding)
```

```
Tunable parameters: memory cost (KB), time cost (iterations), parallelism degree - gen
uinely three independent knobs, allowing much finer-grained tuning to a specific serve
r's available hardware than bcrypt's single cost factor
```

11. PBKDF2 — The Older, Still-Widely-Deployed Standard

Predates bcrypt (1993 vs 1999… actually PBKDF2 is 2000/RFC 2898), still extremely common because it's built into many standard libraries and required by certain compliance frameworks (FIPS 140-2 validated implementations, for instance) — but is **compute-hard only, with no memory-hardness at all**, making it meaningfully weaker against GPU/ASIC cracking than bcrypt, scrypt, or Argon2 at an equivalent iteration count:

```
Format:       varies by implementation, commonly:
              pbkdf2_sha256$[iterations]$[salt]$[hash]
Hashcat mode: -m 10900 (PBKDF2-SHA256, one of several PBKDF2 variants)

Iteration count is the sole tunable cost knob (no memory parameter at all) - modern gu
idance recommends several hundred thousand iterations minimum for SHA-256-based PBKDF
2, a number that keeps climbing as hardware improves, precisely BECAUSE there's no mem
ory-hardness to lean on as a second line of defense
```

Still an acceptable choice when required by compliance constraints, but Argon2id or bcrypt are generally preferred for new systems specifically because of this memory-hardness gap.

12. sha512crypt / sha256crypt — The Common Linux /etc/shadow Default

Referenced in both your JtR and Hashcat guides as a common modern Linux password hash — technically a compute-hardened *construction* built around repeated SHA-512/SHA-256 rounds (not a from-scratch algorithm like bcrypt/Argon2), identifiable by its `$6$` (SHA-512) or `$5$` (SHA-256) prefix in `/etc/shadow`:

```
Format:       $6$[salt]$[hash]     (SHA-512 variant)
Hashcat mode: -m 1800
JtR format:   --format=sha512crypt

Default round count (5,000) is configurable via the salt field itself (embedding a cus
tom round count is supported) - meaningfully harder than raw unsalted SHA-512 (Section
3) due to both the salting AND the round-repetition, but still lacks the deliberate me
mory-hardness of scrypt/Argon2
```

---

13. Comparative Cracking Speed — Putting Every Section Side by Side

```
Roughly ordered fastest (weakest) to slowest (strongest), same modern GPU, illustrativ
e order of magnitude only:

NTLM / MD5 / SHA-1 / SHA-256 (unsalted)  → tens of billions/sec
sha512crypt (5000 rounds, salted)          → tens of millions/sec
PBKDF2-SHA256 (100k+ iterations)             → hundreds of thousands/sec
bcrypt (cost 12)                               → tens of thousands/sec
scrypt (standard params)                         → thousands/sec
```

```
Argon2id (recommended params)                    → low thousands/sec
                                                     or lower
```

This table is the empirical version of Section 1's core claim, and the exact same relationship your Hashcat guide's Section 7 benchmark table gestured at without spelling out every rung of the ladder.

---

14. Identification Methodology

```
1. Look at the hash's structural PREFIX/format first - $2a$/$2b$ (bcrypt), $6$ (sha512
crypt), $argon2id$ (Argon2), a bare 32/40/64/128 hex-char string (MD5/SHA-1/SHA-256/SH
A-512, unsalted and ambiguous without more context)

2. Check the SOURCE system - a Windows SAM/NTDS dump is almost certainly NTLM; a moder
n Linux /etc/shadow entry is almost certainly sha512crypt or a stronger variant; a mod
ern web framework's default is increasingly Argon2id or bcrypt

3. Use hashcat --example-hashes -m X or hashid/hash-identifier tooling to confirm stru
ctural format before committing real compute time - your JtR and Hashcat guides both f
lagged wrong format/mode selection as the single most common reason a cracking session
silently fails

4. If genuinely ambiguous (bare hex string, no metadata), consider digest LENGTH as a
first-pass filter (32=MD5/NTLM, 40=SHA-1, 64=SHA-256, 128=SHA-512) - but remember leng
th alone can't distinguish MD5 from NTLM, since both produce 32 hex characters
```

---

15. Prevention — Choosing and Configuring Hash Algorithms

```
- Use Argon2id for new systems where possible - current best- practice recommendation,
balances GPU and side-channel resistance
- bcrypt remains a solid, well-tested choice where Argon2 isn't available in your stac
k's library ecosystem - vastly preferable to any fast general-purpose hash regardless
- If PBKDF2 is compliance-mandated, use the highest iteration count your legitimate au
thentication latency budget allows, and prefer SHA-256 over SHA-1 as PBKDF2's underlyi
ng function
- NEVER use MD5, SHA-1, or unsalted SHA-256/512 for password storage under any circums
tances, regardless of how "secure" the underlying hash function is for ITS originally
intended purpose (Section 6) - the mismatch between design intent and actual use is th
e entire problem, not the algorithm's general cryptographic soundness
- Always confirm salting is genuinely unique-per-password and handled by the library i
tself, not manually implemented - Section 8's guarantee only holds if the implementati
on is correct
- Periodically re-hash stored passwords at LOGIN time when a weaker legacy algorithm i
```

```
s discovered in an existing system, migrating users to a stronger algorithm transparen
tly as they authenticate rather than requiring a disruptive mass password reset
```

16. Practical Lab Example

```
Directly extends your JtR/Hashcat guides' own closing exercises - this guide is best s
tudied by generating REAL hashes yourself with each algorithm covered above and timing
identical wordlist attacks against each, rather than by reading benchmark numbers.

Suggested practice order:
1. Hash the SAME test password with MD5, bcrypt, and Argon2id (most languages' standar
d/crypto libraries support all three directly - a five-line script in Python is enoug
h)
2. Run an identical wordlist+rules attack (per your Wordlists guide) against all three
resulting hashes, same hardware, and time each - Section 13's table made empirical on
your own machine
3. Inspect each hash's raw string output and identify the salt, cost/iteration paramet
ers, and algorithm prefix by eye (Section 14) before running hashcat --example-hashes
to confirm you read it correctly
```

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-916 (Use of Password Hash With Insufficient Computational Effort - the exact CWE both your JtR and Hashcat guides mapped to, now with its full algorithm-level detail filled in here), CWE-759 (Use of a One-Way Hash without a Salt, covering Section 8 specifically), CWE-327 (Use of a Broken or Risky Cryptographic Algorithm, covering MD5/SHA-1's collision-resistance weaknesses independent of the speed issue)
OWASP Top 10:2025	A02:2025 Security Misconfiguration / Cryptographic Failures - the OWASP Password Storage Cheat Sheet is the direct authoritative source for Section 15's Argon2id-first recommendation
CVSS	Not directly CVSS-scored (algorithm choice is a configuration property) - severity is realized entirely through the cracking feasibility differences quantified in Section 13
Cyber Kill Chain	N/A directly - this guide is foundational/reference material underlying the Actions on Objectives phase covered in your JtR and Hashcat guides

Framework	Mapping
MITRE ATT&CK	T1110.002 (Brute Force: Password Cracking) - identical mapping to both cracking guides, since hash type selection is what determines how FEASIBLE that technique actually is against a given target

Quick Reference Cheat Sheet

```
FAST / GENERAL-PURPOSE (never for passwords):
  MD5          32 hex        -m 0      --format=raw-md5
  SHA-1        40 hex        -m 100    --format=raw-sha1
  SHA-256      64 hex        -m 1400   --format=raw-sha256
  SHA-512      128 hex       -m 1700   --format=raw-sha512
  NTLM         32 hex        -m 1000   --format=nt
  Kerberos TGS varies        -m 13100  --format=krb5tgs

SLOW / PASSWORD-HARDENED (correct choice for storage):
  sha512crypt  $6$...        -m 1800   --format=sha512crypt
  PBKDF2-SHA256 varies       -m 10900  --format=pbkdf2-hmac-sha256
  bcrypt       $2a$/$2b$/... -m 3200   --format=bcrypt
  scrypt       varies        -m 8900   --format=scrypt
  Argon2id     $argon2id$... -m 34000  --format=argon2

IDENTIFICATION QUICK CHECK:
  Prefix $2a$/$2b$/$2y$ → bcrypt        Prefix $6$ → sha512crypt
  Prefix $argon2id$      → Argon2         32/40/64/128 bare hex →
                                              MD5/SHA1/SHA256/SHA512
                                              (check SOURCE system
                                              to disambiguate NTLM vs MD5)

Root lesson: hash function security has TWO independent axes -
cryptographic soundness (collision-resistance, one-wayness) and
computational cost (speed per guess). Part 1's algorithms are sound
on the first axis but fail on the second FOR PASSWORD STORAGE
SPECIFICALLY; Part 2's algorithms were built explicitly to win on
the second axis, which is the only axis that determines cracking
resistance against your JtR/Hashcat guides' techniques.
```